# LABORATORY RECORD

**Course:** Full Stack Web Development
**Course Code:** 23CM4121
**Experiment No.:** 6
**Student Name:** K. Saranya
**Roll No.:** A24126552090
**Date:** 30-08-2026

## 1. TITLE

Implement Modules in Node.js.

## 2. AIM

To understand and implement built-in, user-defined, and third-party modules in Node.js, and demonstrate their usage through file handling, operating system information, path manipulation, HTTP server creation, URL parsing, events, and Express.js.

## 3. OBJECTIVE

The objectives of this experiment are:

- To understand the concept of modules in Node.js.
- To use built-in Node.js modules.
- To perform file operations using the fs module.
- To obtain operating system information using the os module.
- To manipulate file paths using the path module.
- To create a basic HTTP server using the http module.
- To parse URLs using the url module.
- To create and handle custom events using the events module.
- To understand user-defined modules using module.exports.
- To import and use functions from a user-defined module.
- To understand the use of the Express.js module for creating a web server.

## 4. THEORY

Node.js provides a modular architecture that allows an application to be divided into smaller, reusable components called modules. Modules help organise code and make applications easier to maintain and reuse.

Node.js modules can be broadly classified into three types: Built-in Modules, User-defined Modules, and Third-party Modules.

**Built-in Modules**

Node.js provides several built-in modules that can be directly imported using the require() function. The following built-in modules are demonstrated in this experiment:

- fs — used for file system operations such as creating and writing files.
- os — provides information about the operating system and CPU.
- path — provides utilities for working with file and directory paths.
- http — used to create HTTP servers.
- url — used to parse and work with URLs.
- events — provides the EventEmitter class for creating and handling custom events.

**User-defined Module**

A user-defined module is a JavaScript file created by the developer. Functions or variables can be exported using module.exports and imported into another file using require(). In this experiment, arithmetic functions — addition, subtraction, multiplication, and division — are exported from a user-defined module.

**Express Module**

Express.js is a third-party web framework for Node.js that simplifies the process of creating web servers and handling HTTP requests and responses. In the Express program, a GET route is created for /, which sends the message “Hello from Express!” to the browser.

The basic module working flow is:

*Module Creation / Built-in Module → require() → Functionality → Output*

## 5. ALGORITHM / PROCEDURE

1. Create a new Node.js project.
2. Initialize the project using npm init.
3. Import the required built-in modules using require().
4. Use the fs module to create and write data into a text file.
5. Use the os module to display the operating system platform and CPU core count.
6. Use the path module to obtain the base name of a file path.
7. Use the http module to create a basic Node.js HTTP server.
8. Start the HTTP server on port 3000.
9. Use the url module to create and parse a URL.
10. Display the protocol, host, path, and query parameters.
11. Import the EventEmitter class from the events module.
12. Create a custom event using EventEmitter.
13. Register an event listener and trigger the event.
14. Create a user-defined module containing arithmetic functions.
15. Export the functions using module.exports.
16. Import the user-defined module using require().
17. Use the exported functions in another JavaScript file.
18. Install Express.js using npm install express.
19. Create an Express application.
20. Define a GET route and start the Express server.
21. Execute the programs and verify the output.

## 6. PROGRAM

***Program 1: Built-in Modules — builtIn.js***

builtIn.js

```
//fs module
const fs = require("fs");
fs.writeFileSync("hello.txt", "hello node.js");
console.log("File created");

//os module
const os = require("os");
console.log(os.platform());       //os which we are using
console.log(os.cpus().length);    //no. of cpu cores

//path module
const path = require("path");
console.log(path.basename("C:/Users/SARANYA/Desktop/A24126552090/week6/hello.txt"));

// http module
const http = require("http");
const server = http.createServer((req, res) => {
    res.end("Hello from Node.js!");
});
server.listen(3000);
console.log("Server running on port 3000");

// url module
const { URL } = require("url");
const myURL = new URL("https://example.com:8080/products?id=101");
console.log("Protocol:", myURL.protocol);
console.log("Host:", myURL.host);
console.log("Path:", myURL.pathname);
console.log("Query:", myURL.search);

//events module
const EventEmitter = require("events");
const myEvent = new EventEmitter();
myEvent.on("welcome", () => {
    console.log("Welcome to Node.js!");
});
myEvent.emit("welcome");
```

***Program 2: Express Module — module.js***

module.js

```
const express = require("express");
const app = express();

app.get("/", (req, res) => {
    res.send("Hello from Express!");
});

app.listen(3000, () => {
    console.log("Server running on port 3000");
});
```

***Program 3: User-defined Module — app.js***

app.js

```
// exporting user defined module
function add(a, b) {
    return a + b;
}
function sub(a, b) {
    return a - b;
}
function mul(a, b) {
    return a * b;
}
function div(a, b) {
    return a / b;
}

module.exports = {
    add, sub, mul, div
};
```

***Program 3: User-defined Module — user.js***

user.js

```
const math = require("./app");

console.log("Add:", math.add(10, 5));
console.log("Sub:", math.sub(10, 5));
console.log("Mul", math.mul(10, 5));
console.log("Div", math.div(10, 5));
```

**Supporting File: hello.txt**

*The fs module creates the file hello.txt and writes the following content into it: hello node.js*

## 7. CODE EXPLANATION

| Code / Concept | Explanation |
|---|---|
| require("fs") | Imports the built-in File System module. |
| writeFileSync() | Creates or overwrites a file and writes data synchronously. |
| hello.txt | Text file created by the fs module. |
| require("os") | Imports the Operating System module. |
| os.platform() | Returns information about the operating system platform. |
| os.cpus() | Returns information about the available CPU cores. |
| require("path") | Imports the Path module. |
| path.basename() | Returns the final portion of a file path. |
| require("http") | Imports the HTTP module. |
| http.createServer() | Creates an HTTP server. |
| res.end() | Sends a response and ends the HTTP response. |
| server.listen(3000) | Starts the HTTP server on port 3000. |
| require("url") | Imports URL-related functionality. |
| new URL() | Creates a URL object for parsing URL components. |

| Code / Concept | Explanation |
|---|---|
| protocol / host / pathname / search | Return the protocol, hostname and port, path, and query string of the URL. |
| require("events") | Imports the Node.js Events module. |
| EventEmitter | Used to create and manage custom events. |
| .on() / .emit() | Registers an event listener and triggers an event. |
| require("express") | Imports the Express.js framework. |
| express() / app.get() / app.listen() | Creates an Express application, defines a GET route, and starts the server. |
| module.exports | Exports functions or objects from a user-defined module. |
| add() / sub() / mul() / div() | Return the sum, difference, product, and quotient of two numbers. |

## 8. TEST CASES AND EXECUTION DATA

The Node.js programs were executed in the terminal and the functionality of the built-in, user-defined, and Express modules was verified against the output shown below.

| Test Case | Operation / Input | Expected Result | Actual Result | Status |
|---|---|---|---|---|
| TC-01 | Execute builtIn.js | Built-in modules should execute successfully | Program executed successfully | Pass |
| TC-02 | Run fs.writeFileSync() | hello.txt should be created | File created successfully | Pass |
| TC-03 | Check file contents | File should contain hello node.js | Correct content written | Pass |
| TC-04 | Execute os.platform() | OS platform should be displayed | win32 displayed successfully | Pass |
| TC-05 | Execute os.cpus().length | CPU core count should be displayed | CPU count displayed successfully | Pass |
| TC-06 | Execute path.basename() | File name should be displayed | hello.txt displayed | Pass |
| TC-07 | Start HTTP server | Server should run on port 3000 | Server started successfully | Pass |
| TC-08 | Parse the given URL | Protocol, host, path, and query should be displayed | URL components displayed | Pass |
| TC-09 | Emit welcome event | Welcome message should be displayed | Welcome message displayed | Pass |
| TC-10 | Run Express application | Express server should start on port 3000 | Server started successfully | Pass |
| TC-11 | Run user.js with exported arithmetic functions | Functions should perform arithmetic operations correctly | Add, Sub, Mul, Div displayed correctly | Pass |

## 9. OUTPUT

**Output: Node.js Modules Execution**

*The following screenshot shows the terminal execution of the Node.js modules programs. It includes the outputs generated by the built-in modules such as fs, os, path, http, url, and events (builtIn.js), the module server (module.js), and the exported arithmetic functions (user.js).*

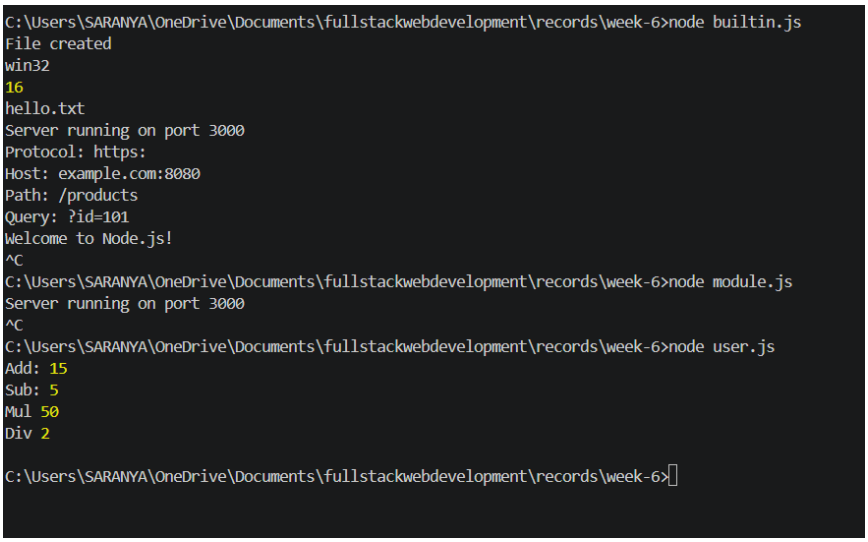


**Figure 1: Output of Modules Implementation Using Node.js**

## 10. RESULT

Thus, the different types of Node.js modules were successfully implemented and executed. Built-in modules such as fs, os, path, http, url, and events were used successfully. A user-defined module was created using module.exports, and the Express.js module was used to create a basic web server. The programs were executed successfully and the expected results were obtained.